

Guatemala, 20 de agosto de 202
Universidad del Valle de Guatemala
Ingeniería en Ciencias de la Computación y Tecnologías de la Información
CC3067 Redes



Proyecto 1

Uso de un protocolo existente

Reporte parcial Entrega 1

Melisa Mendizabal (23778)

Enlace al Repositorio: https://github.com/melisadmendizabal/redes_proyecto1

Alcance de esta entrega

Este documento cubre los incisos 8 (especificación de los servidores MCP implementados) y 10 (conclusiones), correspondientes al avance de la entrega parcial del proyecto, según lo indicado por el catedrático. El inciso 9 (análisis con Wireshark) corresponde a la segunda entrega, una vez implementado el servidor MCP remoto.

Hasta este punto del proyecto, el chatbot (anfitrión) implementado en Python se conecta exitosamente a tres servidores MCP mediante un cliente propio, sin usar SDKs de MCP, hablando JSON-RPC 2.0 manualmente sobre stdio:

- Filesystem MCP Server (oficial, @modelcontextprotocol/server-filesystem)
- Git MCP Server (oficial, mcp-server-git)
- Pharmacy MCP Server (propio, implementado desde cero para este proyecto)

Nota sobre el proveedor del LLM

El enunciado del proyecto sugiere el uso de los LLMs de Anthropic, que ofrecen \$5 de crédito gratuito en su API. Sin embargo, no fue posible activar dicho crédito de prueba: la plataforma de Anthropic requiere verificación telefónica por SMS para habilitarlo, y esta verificación no fue aceptada para el número de teléfono correspondiente a la región (Guatemala). Por esta razón, se utilizó como proveedor del LLM la API de Google Gemini en su lugar.

Este cambio de proveedor no afectó la arquitectura general del proyecto: la comunicación con el LLM está encapsulada en un único módulo (core/llm_client.py), completamente independiente del cliente MCP, del manager de servidores y de la interfaz de consola. Cambiar de proveedor implicó únicamente adaptar ese módulo y el manejo del historial de conversación (core/chatbot.py), sin modificar el resto del sistema.

Especificación de los servidores MCP

Filesystem MCP Server (oficial)

Servidor oficial de Anthropic, ejecutado localmente vía subprocesso (npx). Expone operaciones de manipulación de archivos dentro de un directorio restringido (allowed directory), pasado como argumento al iniciar el servidor.

- Comando de arranque:

```
npx -y @modelcontextprotocol/server-filesystem <ruta_permitida>
```

- Herramientas relevantes utilizadas en la demo de este proyecto:

Herramienta	Parámetros	Descripción
write_file	path (string), content (string)	Crea o sobrescribe un archivo con el contenido indicado.
list_allowed_directories	(sin parámetros)	Lista los directorios donde el servidor tiene permitido operar.

- Ejemplo real de solicitud/respuesta JSON-RPC capturado durante la demo:

```
// Solicitud
{"jsonrpc":"2.0","id":4,"method":"tools/call","params":
  {"name":"write_file","arguments":{
    "content":"Proyecto MCP - CC3067",
    "path":"...\\workspace\\README.md"}}}

// Respuesta
```

```
{
  "jsonrpc": "2.0",
  "id": 4,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Successfully wrote to ...\\workspace\\README.md"
      }
    ]
  }
}
```

Git MCP Server (oficial)

Servidor oficial de Anthropic (paquete de Python mcp-server-git), ejecutado localmente. Expone operaciones de control de versiones sobre un repositorio git indicado por su ruta absoluta (repo_path) en cada llamada.

- Comando de arranque:

```
python -m mcp_server_git --repository <ruta_del_repo>
```

- Herramientas relevantes utilizadas en la demo de este proyecto:

Herramienta	Parámetros	Descripción
git_status	repo_path (string)	Muestra el estado del árbol de trabajo del repositorio.
git_add	repo_path (string), files (array de strings)	Agrega archivos al área de staging.
git_commit	repo_path (string), message (string)	Registra un commit con los cambios en staging.

- Ejemplo real de solicitud/respuesta JSON-RPC capturado durante la demo:

```
// Solicitud
{
  "jsonrpc": "2.0",
  "id": 5,
  "method": "tools/call",
  "params": {
    "name": "git_commit",
    "arguments": {
      "repo_path": "...\\workspace",
      "message": "initial commit"
    }
  }
}

// Respuesta
{
  "jsonrpc": "2.0",
  "id": 5,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "Changes committed successfully with hash 838e4ab6..."
      }
    ],
    "isError": false
  }
}
```

Pharmacy MCP Server (propio)

Servidor MCP implementado desde cero para este proyecto (sin SDK de MCP), correspondiente al caso de uso de industria: una cadena de farmacias que recomienda y vende medicamentos según los síntomas del cliente. Se ejecuta localmente vía stdio, hablando JSON-RPC 2.0 a mano (framing por línea, ciclo de vida initialize, luego notifications/initialized, después tools/list y finalmente tools/call).

- Comando de arranque:

```
python -m servers.pharmacy.local_stdio
```

- Catálogo de datos:

el servidor mantiene en memoria un catálogo de 4 medicamentos (paracetamol, ibuprofeno, loratadina, omeprazol), cada uno con descripción, síntomas asociados, precio y stock. El stock se actualiza en tiempo real conforme se realizan compras durante la sesión.

- Herramientas expuestas:

Herramienta	Parámetros	Descripción
search_by_symptom	symptom (string, requerido)	Busca y recomienda medicamentos según un síntoma descrito en lenguaje natural.
get_medication_info	name (string, requerido)	Regresa descripción, precio y stock de un medicamento específico.
list_medications	(sin parámetros)	Lista el catálogo completo de medicamentos disponibles.
purchase_medication	name (string, requerido), quantity (integer ≥ 1 , requerido)	Realiza la compra de una cantidad de un medicamento, validando stock disponible.

- Ejemplo de uso, flujo real capturado durante pruebas (conversación con contexto mantenido entre turnos):

Usuario: "tengo dolor de cabeza, ¿qué me recomiendas?"

```
// Solicitud MCP
{"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": {
  "name": "search_by_symptom", "arguments": {"symptom": "dolor de cabeza"}}}
```

```
// Respuesta MCP
{"jsonrpc": "2.0", "id": 3, "result": {"content": [{"type": "text",
  "text": "Para 'dolor de cabeza' se recomienda: paracetamol,
  ibuprofeno."}],
  "isError": false}}
```

Bot: "Para el dolor de cabeza se recomiendan: Paracetamol (Q15.00, 50 en stock) e Ibuprofeno (Q20.00, 40 en stock)..."

Usuario: "cómprame 2 de esos"

Bot: "¿Te refieres a 2 de Paracetamol, 2 de Ibuprofeno o 2 de cada uno?"

Usuario: "uno de cada uno"

```
// Solicitud MCP
{"jsonrpc": "2.0", "id": 6, "method": "tools/call", "params": {
```

```
  "name": "purchase_medication", "arguments": {"name": "paracetamol", "quantity": 1}}}
```

```
// Respuesta MCP
{"jsonrpc": "2.0", "id": 6, "result": {"content": [{"type": "text",
  "text": "Compra confirmada: 1 x paracetamol = Q15.00. Stock
  restante: 49."}],
  "isError": false}}
```

Bot: "¡Compra realizada con éxito! 1 x Paracetamol: Q15.00 (stock

restante 49), 1 x Ibuprofeno: Q20.00 (stock restante 39).
Total: Q35.00"

Este ejemplo evidencia tanto el correcto funcionamiento del servidor MCP propio como el mantenimiento de contexto conversacional: en el segundo turno el usuario no repite los nombres de los medicamentos ('cómprame 2 de esos'), y el modelo resuelve la referencia correctamente contra el turno anterior, incluso pidiendo aclaración ante la ambigüedad en la cantidad.

Conclusiones y comentarios (avance parcial)

- A la fecha de esta entrega parcial, se ha logrado implementar y validar de forma funcional: la conexión con un LLM a nivel de API (Google Gemini) manteniendo contexto de sesión; un cliente MCP propio, sin SDKs, que implementa el framing de JSON-RPC 2.0 sobre stdio (mensajes delimitados por saltos de línea) y el ciclo de vida completo del protocolo (initialize, notifications/initialized, tools/list, tools/call); la integración con los servidores MCP oficiales de Filesystem y Git; y el desarrollo de un servidor MCP propio para el caso de uso de una cadena de farmacias, con cuatro herramientas funcionales.
- Una de las principales dificultades encontradas fue de infraestructura más que de protocolo: en Windows, subprocess.Popen no resuelve automáticamente ejecutables de tipo .cmd (como npx), lo cual requirió resolver la ruta del ejecutable manualmente antes de lanzar el proceso. Asimismo, se debió ajustar el límite de iteraciones del ciclo de function calling, ya que el modelo de lenguaje utilizado no siempre agrupa varias llamadas a herramientas en un mismo turno, sino que las solicita de forma secuencial, consumiendo más vueltas de las inicialmente previstas.
- Otra dificultad relevante fue el cambio de proveedor de LLM a mitad de desarrollo (de Anthropic a Google Gemini, por restricciones de acceso a créditos gratuitos), lo cual implicó adaptar el formato de mensajes (roles user/model en lugar de user/assistant) y el formato de function calling (functionDeclarations/functionCall/functionResponse). La arquitectura en capas del proyecto, separando el cliente del LLM (llm_client.py) del resto de la lógica del chatbot, permitió realizar este cambio afectando únicamente dos archivos, sin modificar el cliente MCP, el manager, ni la interfaz de consola.
- Como lección aprendida, se destaca la importancia de robustecer el cliente del LLM ante errores transitorios de la API (límites de cuota HTTP 429 y sobrecarga del servidor HTTP 503), implementando reintentos automáticos con espera, en lugar de asumir que cada llamada a la API será exitosa.
- Para la siguiente entrega, se planea: exponer el servidor de farmacia también mediante transporte HTTP (Streamable HTTP) reutilizando la misma lógica de negocio, desplegarlo en Google Cloud Run; y realizar el análisis de tráfico con Wireshark sobre la comunicación remota.